

METAHEURÍSTICAS DE BUSCA LOCAL

VND, VNS E GRASP

DCE770 - Heurísticas e Metaheurísticas

Atualizado em: 13 de setembro de 2026

Iago Carvalho

Departamento de Ciência da Computação



A BUSCA LOCAL PAROU. E AGORA?

Na aula anterior, uma busca por descida terminou sem vizinho viável aprimorante.

Nesta aula, vamos comparar três decisões:

1. **Recomeçar:** construir outra solução inicial (Multi-start e GRASP)
2. **Mudar os movimentos:** explorar outras vizinhanças (VND)
3. **Perturbar e refinar:** buscar a partir de outra região (VNS)

Ao final, deveremos conseguir executar esses métodos passo a passo e justificar as escolhas de construção, vizinhança, aceitação e parada.

Manteremos a notação utilizada na aula anterior:

- Ω : espaço de busca; $\mathcal{F} \subseteq \Omega$: soluções viáveis
- f : função objetivo a ser minimizada
- s_0 : solução inicial; s : corrente; s' : candidata
- $\tilde{s}$: solução perturbada; s^* : melhor solução encontrada
- $N_i(s)$: i -ésima vizinhança; $N_{i,\mathcal{F}}(s) = N_i(s) \cap \mathcal{F}$

Para uma família ordenada de r vizinhanças, escreveremos

$$\mathcal{N}_{\mathcal{F}} = (N_{1,\mathcal{F}}, \dots, N_{r,\mathcal{F}}).$$

Uma heurística também pode ser implementada como um algoritmo.

- **Método exato:** encontra uma solução ótima e certifica sua otimalidade quando conclui a resolução de uma instância viável
- **Heurística:** procura boas soluções, em geral sem garantia de otimalidade
- **Metaheurística:** estratégia geral que orienta a construção de heurísticas para diferentes problemas

Para resolver um problema concreto, precisamos definir representação, movimentos, avaliação e tratamento das restrições.

POR QUE USAR METAHEURÍSTICAS?

Em muitos problemas de otimização combinatória, enumerar todas as soluções candidatas é impraticável.

- Precisamos obter boas soluções dentro de um orçamento computacional
- Métodos exatos e heurísticos podem ser usados de forma complementar
- A escolha depende da instância, do tempo disponível e da garantia exigida

Metaheurísticas exploram o espaço de busca de forma **guiada**, combinando decisões determinísticas e, frequentemente, aleatórias.

O QUE ESPERAMOS DE UMA BOA IMPLEMENTAÇÃO?

- **Clareza:** componentes e regras de busca bem definidos
- **Generalidade:** possibilidade de adaptar a estratégia a outros problemas
- **Eficácia:** soluções de boa qualidade nas instâncias de interesse
- **Eficiência:** bom uso do tempo e das avaliações disponíveis
- **Robustez:** resultados consistentes em diferentes execuções e instâncias

Essas são metas de projeto, **não garantias universais**

- A qualidade deve ser verificada experimentalmente.

Diversificação

- Explorar regiões diferentes do espaço de busca
- Nesta aula: reinícios e perturbações

Intensificação

- Explorar com mais cuidado uma região promissora
- Nesta aula: busca por descida e combinação de vizinhanças no VND

DA BUSCA POR DESCIDA ÀS METAHEURÍSTICAS

Na aula passada estudamos as buscas pelo primeiro e pelo melhor aprimorante:

- Aceitam apenas melhorias estritas de f
- Ao esgotar a vizinhança viável, encontram um ótimo local relativo a ela
- Não garantem que esse ótimo local seja global
- Em um espaço finito, a melhoria estrita impede revisitar soluções

Usaremos *BuscaLocal* para representar essa busca por descida. O nome *Hill Climbing* corresponde à perspectiva de maximizar a qualidade.

MULTI-START

O Multi-start executa diversas vezes uma busca local.

- Cada execução gera uma solução inicial, que pode coincidir com uma anterior
- Podemos usar, por exemplo, uma construção gulosa semialeatória
- Os reinícios funcionam como um mecanismo de diversificação

Critérios de parada possíveis:

- Orçamento de tempo ou de avaliações
- Número de reinícios, eventualmente sem melhora da melhor solução

Algoritmo 1: Multi-start

Entrada: $N_{\mathcal{F}}$ // Vizinhança viável

Entrada: f // Função objetivo a ser minimizada

1 $s_0 \leftarrow$ solução inicial gulosa semialeatória

2 $s^* \leftarrow \text{BuscaLocal}(s_0, N_{\mathcal{F}}, f)$

3 **enquanto** *critério de parada não atingido* **faça**

4 $s_0 \leftarrow$ solução inicial gulosa semialeatória

5 $s \leftarrow \text{BuscaLocal}(s_0, N_{\mathcal{F}}, f)$

6 **se** $f(s) < f(s^*)$ **então**

7 $s^* \leftarrow s$ // Atualiza a melhor solução

8 **retorne** s^*

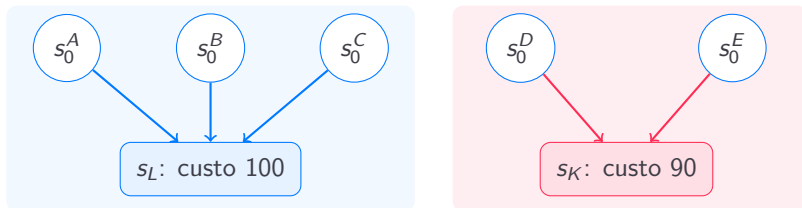
MULTI-START: REINICIAR NÃO É CONTINUAR A TRAJETÓRIA

Cada execução completa da busca local encontra um ótimo local, mas execuções diferentes podem chegar ao **mesmo** ótimo.

- O reinício abandona a trajetória anterior e gera outro ponto de partida
- Na versão básica, guardamos a melhor solução, sem orientar a construção com o histórico das demais soluções
- A diversidade dos pontos iniciais influencia as regiões visitadas

MULTI-START: BACIAS DE ATRAÇÃO

Fixadas a vizinhança, a regra de escolha e o desempate determinístico, uma bacia reúne os pontos iniciais que levam ao mesmo ótimo local.



Exemplo esquemático: cinco reinícios, mas apenas dois ótimos distintos.

VARIABLE NEIGHBORHOOD DESCENT (VND)

O VND integra a família de métodos de vizinhança variável de Mladenović e Hansen.

- Mladenović e Hansen (1997). *Variable neighborhood search*. [▶ Referência](#)
- Uma revisão mais ampla: [▶ Revisão](#)

Explora diversas estruturas de vizinhança de forma iterativa.

- Uma solução ótima para uma vizinhança pode não ser ótima para outra
- Combinar vizinhanças pode levar a ótimos locais de melhor qualidade

O VND opera com uma família ordenada $\mathcal{N}_{\mathcal{F}} = (N_{1,\mathcal{F}}, \dots, N_{r,\mathcal{F}})$.

- Uma opção é ordenar pelo custo de exploração, das mais baratas às mais caras
- Também importa a capacidade de encontrar melhorias complementares
- Vizinhanças não precisam ser encaixadas nem ter tamanhos crescentes

Essa ordem é uma decisão de projeto, não uma exigência do método.

No VND sequencial desta aula, uma melhora reinicia a exploração em $i = 1$.

O VND pode ser resumido em quatro passos:

1. **Inicialização:** obtém $s \in \mathcal{F}$, define a família e faz $i \leftarrow 1$
2. **Busca:** aplica busca local a s utilizando $N_{i,\mathcal{F}}$
3. **Movimento:** se s' melhora s , faz $s \leftarrow s'$ e $i \leftarrow 1$; caso contrário, $i \leftarrow i + 1$
4. **Término:** para quando $i > r$, após percorrer todas as vizinhanças sem alterar a solução corrente

Com buscas locais completas, esse término certifica otimalidade local em todas as r vizinhanças. Uma interrupção por tempo **não** oferece essa garantia.

Algoritmo 2: Variable Neighborhood Descent

Entrada : $\mathcal{N}_{\mathcal{F}}$ // Família ordenada de vizinhanças viáveis
Entrada : f // Função objetivo a ser minimizada
Entrada : $s \in \mathcal{F}$ // Solução inicial

```
1  $i \leftarrow 1$ 
2  $r \leftarrow |\mathcal{N}_{\mathcal{F}}|$ 
3 enquanto  $i \leq r$  faça
4    $s' \leftarrow \text{BuscaLocal}(s, N_{i,\mathcal{F}}, f)$ 
5   se  $f(s') < f(s)$  então
6      $s \leftarrow s'$  // Atualiza a solução corrente
7      $i \leftarrow 1$  // Reinicia as vizinhanças
8   caso contrário
9      $i \leftarrow i + 1$ 
10 retorne  $s$ 
```

VND: ESCOLHENDO VIZINHANÇAS PARA O PRV

Retomemos o problema de roteamento de veículos capacitado da aula anterior

Uma família possível é:

- $N_{1,\mathcal{F}}$: movimentos *2-opt* dentro de uma rota
- $N_{2,\mathcal{F}}$: transferência de um cliente (*relocate*)
- $N_{3,\mathcal{F}}$: troca de clientes entre rotas (*swap*)

Cada movimento deve manter o atendimento dos clientes e respeitar as capacidades.
Essa ordem é ilustrativa: seu desempenho deve ser medido.

Por exemplo, o *relocate* pode mudar a distribuição de clientes e abrir novas oportunidades para o *2-opt* dentro das rotas.

VND: EXECUÇÃO PASSO A PASSO

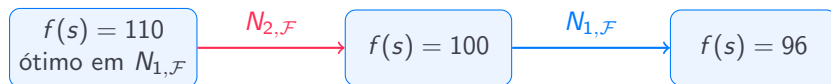
Exemplo didático de custos retornados pelas buscas locais, com $r = 2$. Cada chamada termina em um ótimo local da vizinhança utilizada.

Chamada	i	$f(s)$ antes	$f(s')$	Decisão	Próximo i
1	1	120	110	Aceitar	1
2	1	110	110	Trocar vizinhança	2
3	2	110	100	Aceitar e reiniciar	1
4	1	100	96	Aceitar	1
5	1	96	96	Trocar vizinhança	2
6	2	96	96	Encerrar	3

A solução de custo 110 era ótima em $N_{1,\mathcal{F}}$, mas não em $N_{2,\mathcal{F}}$.

A solução final é ótima em ambas.

POR QUE VOLTAR À PRIMEIRA VIZINHANÇA?



- A ausência de melhorias foi verificada para a **solução anterior**
- Ao mudar s , mudam também os conjuntos $N_{i,F}(s)$
- Portanto, precisamos testar novamente as vizinhanças anteriores

Pergunta: se o tempo acabar após a chamada 4, o que podemos garantir?

Ao terminar uma passagem completa sem melhoria:

$$f(s) \leq f(u), \quad \forall u \in \bigcup_{i=1}^r N_{i,\mathcal{F}}(s).$$

- O VND pode sair de um ótimo local de uma vizinhança usando outra
- Não sai, por melhoria estrita, de um ótimo local comum a todas elas
- A solução obtida depende do ponto inicial e da ordem de exploração
- Combinar vizinhanças não garante otimalidade global

Podemos usar o VND como intensificação dentro de um Multi-start ou de um VNS.

- **Sequencial com reinício:** volta à primeira vizinhança após melhorar
- **Pipe:** permanece na vizinhança após melhorar e avança quando falha
- **Cíclica:** percorre as vizinhanças em ciclos
- **Aleatória:** sorteia a ordem de exploração

Se cada chamada já executa uma descida completa, repetir imediatamente a mesma vizinhança não encontra nova melhoria. Isso difere de aplicar apenas um movimento.

Para certificar otimalidade local conjunta, é preciso verificar todas as vizinhanças para a solução corrente, sem alterações entre essas verificações.

VARIABLE NEIGHBORHOOD SEARCH (VNS)

VARIABLE NEIGHBORHOOD SEARCH - VNS

O VNS combina mudanças sistemáticas de vizinhança, perturbações e busca local.

O *shake* é uma perturbação aleatória da solução corrente.

- Produz uma solução temporária $\tilde{s}$
- A busca local refina $\tilde{s}$ e produz s'
- A regra de aceitação decide se s' substitui s

Assim, podemos tentar sair da região de um ótimo local sem reconstruir uma solução do zero.

O VNS básico pode ser resumido em cinco passos:

1. **Inicialização:** obtém $s \in \mathcal{F}$ e faz $i \leftarrow 1$
2. **Shake:** sorteia $\tilde{s} \in N_{i,\mathcal{F}}(s)$
3. **Busca:** aplica busca local a $\tilde{s}$ e obtém s'
4. **Aceitação:** se $f(s') < f(s)$, faz $s \leftarrow s'$ e $i \leftarrow 1$; caso contrário, $i \leftarrow i + 1$
5. **Repetição:** ao esgotar as r vizinhanças, inicia outro ciclo, enquanto houver orçamento

O critério de parada pode ser tempo, avaliações ou tentativas sem melhoria.

Algoritmo 3: Variable Neighborhood Search

Entrada : $\mathcal{N}_{\mathcal{F}}$, com r vizinhanças // Família ordenada
Entrada : $f, s \in \mathcal{F}$ // Objetivo a minimizar e solução inicial
Entrada : L // Busca local viável, iniciada na solução recebida

```

1 enquanto critério de parada não atingido faça
2    $i \leftarrow 1$ 
3   enquanto  $i \leq r$  e critério de parada não atingido faça
4     se  $N_{i,\mathcal{F}}(s) = \emptyset$  então  $i \leftarrow i + 1$ ; continue
5      $\tilde{s} \leftarrow \text{Shake}(s, N_{i,\mathcal{F}})$ 
6      $s' \leftarrow L(\tilde{s})$ 
7     se  $f(s') < f(s)$  então
8        $s \leftarrow s'$ ;  $i \leftarrow 1$ 
9     caso contrário
10       $i \leftarrow i + 1$ 
11 retorne  $s$ 
  
```

As vizinhanças do *shake* e as da busca local têm papéis diferentes.

- $N_{i,\mathcal{F}}$ controla a perturbação da solução corrente
- L refina a solução perturbada usando f e preservando a viabilidade
- L pode ser uma busca por descida com uma vizinhança fixa
- Outra opção é usar VND com uma família própria de vizinhanças

A busca local deve partir de $\tilde{s}$. Reiniciar independentemente dela desfaz a ligação entre a perturbação e o refinamento.

VNS: PERTURBAR NÃO É ACEITAR A PERTURBAÇÃO

Exemplo didático de custos em duas tentativas consecutivas:

i	$f(s)$	$f(\tilde{s})$	$f(s')$	Decisão	Próximo i
1	100	108	100	Rejeitar	2
2	100	125	92	Aceitar	1

- O shake pode piorar o custo da solução temporária
- A comparação é feita **depois** da busca local: $f(s') < f(s)$
- Na primeira tentativa, a solução corrente permanece com custo 100
- Na segunda, a solução corrente passa a ter custo 92

COMO ESCOLHER A INTENSIDADE DO SHAKE?

No PRV, podemos testar perturbações com diferentes números de movimentos viáveis de relocate ou swap, seguidos do mesmo VND.

- **Pequena:** pode retornar frequentemente ao mesmo ótimo local
- **Grande:** pode explorar outras regiões, mas exigir mais refinamento
- **Intermediária:** deve ser avaliada, não presumida como a melhor

Verifique capacidades a cada movimento. Se não houver movimento viável na vizinhança escolhida, avance para a próxima.

Experimento: compare a frequência de retorno ao mesmo ótimo, o ganho após a busca local e o tempo por tentativa.

Reduced VNS (RVNS)

- Omite a busca local: avalia diretamente o vizinho sorteado
- Aceita se houver melhoria; caso contrário, muda de vizinhança

Skewed VNS (SVNS)

- Modifica a aceitação levando em conta também a distância entre soluções
- Pode aceitar uma candidata final de pior custo para favorecer diversificação
- Nesse caso, a melhor solução s^* deve ser guardada separadamente de s

Variar a intensidade do shake faz parte do projeto das vizinhanças; não implica, por si só, permitir pioras na solução corrente.

GREEDY RANDOMIZED ADAPTIVE
SEARCH PROCEDURE
(GRASP)

GREEDY RANDOMIZED ADAPTIVE SEARCH PROCEDURE

Feo e Resende apresentam o GRASP na referência clássica de 1995:

- Feo, T. A.; Resende, M. G. C. *Greedy Randomized Adaptive Search Procedures*. Journal of Global Optimization, 6, 109–133. [▶ Artigo](#)

Cada iteração tem duas fases:

1. Construção gulosa, aleatorizada e adaptativa
2. Refinamento por busca local

O GRASP é uma forma de Multi-start com uma construção específica.

Algoritmo 4: GRASP

```
Entrada :  $\mathcal{N}_{\mathcal{F}}$  // Vizinhos usados pela busca local
Entrada :  $f$  // Função objetivo a ser minimizada
Entrada :  $\alpha \in [0, 1]$  // Parâmetro do limiar da LCR
1  $s_0 \leftarrow \text{ConstrucaoGulosaAleatoria}(\alpha)$ 
2  $s^* \leftarrow \text{BuscaLocal}(s_0, \mathcal{N}_{\mathcal{F}}, f)$ 
3 enquanto critério de parada não atingido faça
4    $s_0 \leftarrow \text{ConstrucaoGulosaAleatoria}(\alpha)$ 
5    $s \leftarrow \text{BuscaLocal}(s_0, \mathcal{N}_{\mathcal{F}}, f)$ 
6   se  $f(s) < f(s^*)$  então
7      $s^* \leftarrow s$  // Atualiza a melhor solução
8 retorne  $s^*$ 
```

Uma solução parcial p é ampliada a cada passo:

- **Gulosa**: avalia os candidatos por uma função $g(c \mid p)$
- **Aleatorizada**: sorteia um candidato de uma lista restrita (LCR)
- **Adaptativa**: atualiza candidatos e avaliações após cada inclusão

Aqui, **adaptativa não significa ajustar** α : mesmo com α fixo, a LCR é reconstruída conforme p muda.

A aleatoriedade permite diversidade, mas não garante uma solução diferente a cada iteração.

Uma **lista de candidatos restrita (LCR)** reúne as opções consideradas promissoras para a **próxima decisão de construção**.

- $C(p)$ contém todos os candidatos admissíveis para ampliar a solução parcial p
- A LCR filtra esses candidatos usando a avaliação gulosa $g(c \mid p)$
- Sorteamos **um** elemento da LCR e o incorporamos à solução parcial

No PCV: os candidatos são cidades ainda não visitadas, não rotas completas. A LCR pode conter várias cidades, mas apenas uma será a próxima cidade da rota.

O filtro dá preferência às boas opções; o sorteio permite variar as decisões.

COMO O PARÂMETRO α DEFINE A LCR?

Considere $C(p) \neq \emptyset$. Nesta convenção, **menor g é melhor**.

$$g_{\min} = \min_{c \in C(p)} g(c \mid p), \quad g_{\max} = \max_{c \in C(p)} g(c \mid p).$$

Definimos um limiar de admissão τ :

$$\tau = g_{\min} + \alpha(g_{\max} - g_{\min}), \quad \alpha \in [0, 1].$$

$$\text{LCR}(p, \alpha) = \{c \in C(p) : g(c \mid p) \leq \tau\}.$$

- $g_{\max} - g_{\min}$ é a diferença entre a pior e a melhor avaliação atual
- α determina quanto dessa diferença toleramos acima do melhor valor

GRASP NO PCV: UMA PEQUENA INSTÂNCIA

Exemplo didático de PCV simétrico completo, sem supor distâncias euclidianas.
Partimos de A e acrescentamos uma cidade ainda não visitada ao fim do caminho.

	A	B	C	D	E
A	0	4	7	10	16
B	4	0	5	12	8
C	7	5	0	6	9
D	10	12	6	0	3
E	16	8	9	3	0

- p : caminho parcial; v : sua última cidade
- $C(p)$: cidades ainda não visitadas
- Avaliação gulosa: $g(c \mid p) = c_{v,c}$
- Ao final, fechamos a rota retornando a A

Essa avaliação é míope: considera a próxima aresta, não o custo total da rota futura.

LCR NO PCV: PRIMEIRO FILTRAR, DEPOIS SORTEAR

Partindo de A , temos $g_{\min} = 4$, $g_{\max} = 16$ e $\alpha = 0,25$.

$$\tau = 4 + 0,25(16 - 4) = 7.$$

Candidato	Custo	Entra na LCR?	Probabilidade de escolha
B	4	Sim: $4 \leq 7$	$1/2$
C	7	Sim: $7 \leq 7$	$1/2$
D	10	Não: $10 > 7$	0
E	16	Não: $16 > 7$	0

Na versão com sorteio uniforme, B e C têm a mesma chance, embora B seja mais barato. A preferência gulosa já foi aplicada no filtro.

Escolher C é permitido: o GRASP não exige escolher sempre o candidato de menor custo.

O QUE α CONTROLA — E O QUE NÃO CONTROLA

Para a mesma solução parcial e as mesmas avaliações:

- $\alpha = 0$: entram apenas os melhores candidatos, incluindo empates
- $0 < \alpha < 1$: admitimos também candidatos próximos do melhor valor
- $\alpha = 1$: todos os candidatos admissíveis podem ser sorteados

Atenção: $\alpha = 0,25$ não significa escolher os 25% melhores candidatos, nem escolher aleatoriamente em 25% das decisões.

α controla o **limiar de custo**, não o tamanho fixo da lista. Neste exemplo, $\alpha = 0,25$ admite duas das quatro cidades.

Aumentar α pode ampliar a LCR, mas não garante uma solução final melhor.

GRASP NO PCV: CONSTRUÇÃO COM $\alpha = 0,25$

Uma sequência possível de sorteios é C, D, E, B :

p	Candidatos e custos	Limiar	LCR	Sorteado
A	$B : 4, C : 7, D : 10, E : 16$	7	$\{B, C\}$	C
A, C	$B : 5, D : 6, E : 9$	6	$\{B, D\}$	D
A, C, D	$B : 12, E : 3$	5,25	$\{E\}$	E
A, C, D, E	$B : 8$	8	$\{B\}$	B

$$s_0 = (A, C, D, E, B, A), \quad f(s_0) = 7 + 6 + 3 + 8 + 4 = 28.$$

Observe que $g(B \mid A) = 4$, mas $g(B \mid A, C) = 5$: a avaliação muda com a solução parcial, mesmo mantendo α fixo.

POR QUE RECONSTRUIR A LCR APÓS CADA ESCOLHA?

No primeiro passo, $p = (A)$ e $LCR = \{B, C\}$. Suponha que o sorteio escolha C .

1. A solução parcial passa a ser $p = (A, C)$
2. C deixa de ser candidata, pois já foi visitada
3. Reavaliamos as cidades restantes a partir de C : $g(B \mid p) = 5$, $g(D \mid p) = 6$ e $g(E \mid p) = 9$
4. Recalculamos $\tau = 5 + 0,25(9 - 5) = 6$ e obtemos a nova $LCR = \{B, D\}$

Não basta remover C da lista antiga: D agora também é uma opção elegível.

Esse ciclo de **avaliar**, **filtrar**, **sortear** e **atualizar** se repete até completar a construção. Depois, começa a busca local.

ATIVIDADE: QUAL É A LCR?

No primeiro passo, os custos são $B : 4$, $C : 7$, $D : 10$ e $E : 16$.

1. Determine a LCR para $\alpha = 0$, $0,5$ e 1 .
2. Com $\alpha = 0,25$, escolher C é um erro da construção?
3. Se todos os candidatos tiverem o mesmo custo, o que ocorre com a LCR?
4. Aumentar α garante uma rota final melhor?

Discussão: a qualidade deve ser medida depois da construção e também depois da busca local.

A busca local parte da solução construída $s_0 \in \mathcal{F}$.

- No PCV, podemos usar 2-opt com primeiro ou melhor aprimorante
- No PRV, podemos combinar vizinhanças por meio do VND
- A construção e o refinamento devem usar critérios de viabilidade compatíveis

No exemplo, a construção produziu custo 28. A busca local verifica se há movimentos aprimorantes; não precisamos presumir que eles existam.

Construções com menor custo inicial não necessariamente produzem os melhores resultados depois do refinamento.

GRASP E MULTI-START: QUAL É A RELAÇÃO?

O Multi-start define a estrutura de reinícios; o GRASP especifica uma maneira de construir os pontos iniciais.

- Construção gulosa aleatorizada, com avaliações atualizadas a cada passo
- Busca local iniciada na solução construída
- Armazenamento da melhor solução entre as iterações

Apenas adicionar um parâmetro aleatório a um Multi-start não define, por si só, uma construção GRASP.

- A construção deve retornar $s_0 \in \mathcal{F}$ antes da busca local
- Se uma construção parcial ficar sem continuação admissível, precisamos reparar, retroceder ou reiniciar; não sortear de uma LCR vazia
- Defina orçamento de tempo ou de avaliações para a execução inteira
- Verifique o orçamento também dentro das rotinas longas de busca local
- Se a busca local for interrompida, devolva a melhor solução viável disponível, sem afirmar que ela é um ótimo local

Os pseudocódigos supõem uma inicialização viável e orçamento suficiente para obtê-la. Limites rígidos precisam ser compartilhados por todas as rotinas.

MÉTODOS DE ESCOLHA DE α NO GRASP

- **Fixo:** utiliza o mesmo valor em todas as construções
- **Aleatório:** sorteia um valor a cada iteração, sem usar o histórico
- **Reativo:** adapta probabilidades de escolha a partir do desempenho observado para diferentes valores

Uma estratégia reativa registra os resultados por valor de α e atualiza periodicamente as probabilidades, favorecendo os valores de melhor desempenho. É importante manter oportunidades de exploração.

Não confundir essa adaptação **entre iterações** com a atualização da LCR **dentro de cada construção**, presente no GRASP básico.

COMPARAÇÃO E SÍNTESE

COMO COMPARAR OS MÉTODOS?

1. Use as mesmas instâncias, restrições e condições computacionais
2. Fixe um orçamento comparável: tempo ou número de avaliações
3. Execute várias sementes e registre-as para reprodução
4. Reporte melhor custo, média ou mediana, dispersão e tempo
5. Separe as instâncias de ajuste das instâncias de avaliação

Para uma referência positiva f_{ref} :

$$\text{desvio}(\%) = 100 \frac{f(s) - f_{\text{ref}}}{f_{\text{ref}}}.$$

Se a referência não for um ótimo comprovado, esse desvio não certifica a distância ao ótimo.

SÍNTESE: O QUE MUDA EM CADA MÉTODO?

Método	Ponto de partida	Exploração	Melhor solução
Multi-start	Nova construção	Busca local em cada reinício	Guardada entre reinícios
VND	Solução corrente	Troca de vizinhanças por descida	Corrente nunca piora
VNS básico	Perturbação da corrente	Shake seguido de busca local	Corrente só aceita melhorias
GRASP	Construção gulosa aleatorizada adaptativa	Busca local em cada construção	Guardada entre iterações

Esses componentes podem ser combinados: por exemplo, GRASP com VND.

ATIVIDADE FINAL: PROJETANDO UMA HEURÍSTICA PARA O PRV

Proponham uma combinação dos componentes estudados:

1. Como construir uma solução inicial viável e introduzir diversidade?
2. Quais movimentos usar e em que ordem explorá-los?
3. Como perturbar uma solução sem violar as capacidades?
4. Quando aceitar uma candidata e quando atualizar s^* ?
5. Como comparar a proposta com uma busca local simples?

REFERÊNCIAS PARA APROFUNDAMENTO

- Mladenović, N.; Hansen, P. (1997). *Variable neighborhood search*. Computers & Operations Research, 24(11), 1097–1100. [▶ Artigo](#)
- Hansen, P.; Mladenović, N.; Pérez, J. A. M. (2010). *Variable neighbourhood search: methods and applications*. Annals of Operations Research, 175, 367–407. [▶ Revisão](#)
- Feo, T. A.; Resende, M. G. C. (1995). *Greedy Randomized Adaptive Search Procedures*. Journal of Global Optimization, 6, 109–133. [▶ Artigo](#)
- Pitsoulis, L. S.; Resende, M. G. C. (2001). *Greedy Randomized Adaptive Search Procedures*. Relatório técnico. [▶ Tutorial](#)